

Technical Report: Credit Scoring Model Evaluation and Data Quality Audit

Denis Mwenda Koome

MSc Data Science and Analytics, Strathmore University | DSA 8401: Applied Machine Learning

Prepared for the Head of Credit Risk
August 16, 2026

Executive Summary

This report provides the Head of Credit Risk with a comprehensive evaluation of the data preparation, feature engineering, and MLOps pipeline architecture for the proposed thin-file mobile-money credit scoring model. Our audit identified critical pathologies in the raw extract of 183,600 transactions, most notably two instances of target leakage that falsely inflated initial offline metrics to a perfect 1.0000 ROC-AUC. By implementing a structurally secure, group-aware Scikit-Learn Pipeline keyed on resolved customer identities, we eliminated these leaks, resulting in a reliable, production-safe baseline ROC-AUC of 0.6250.

Section 1. Data Quality Audit

A comprehensive audit of the dataset revealed ten significant data quality issues across the 18,3600 transactions. Each anomaly was evaluated, classified, and remediated as detailed below.

Variable	Issue, Evidence & Missing Class	Remediation Strategy
txn_id	Uniqueness: 3,600 exact duplicate rows. <i>Class:</i> N/A	Deduplication: Stripped prior to data splitting using <code>.drop_duplicates()</code> to prevent train-test contamination.
amount	Validity: Stored as strings with symbols (/-, Dr, parentheses). <i>Class:</i> N/A	Parsing: Applied regex to extract numeric floats, assigning negative signs to debits and positive to credits.
txn_time	Validity: Mixes three distinct formats (ISO, dd/mm/yyyy, Mon dd, yyyy). <i>Class:</i> N/A	Iterative Parsing: Converted all dates sequentially into a standard datetime format ensuring all timestamps precede scoring.
agent_id	Completeness: 13.69% missing, fully explained by region (45% in Mwanza/Jinja/Arusha). <i>Class:</i> MAR	Constant Imputation: Utilized a constant "Unknown" fill value to retain state without dropping rows.
device_id	Completeness: 5.91% missing uniformly. <i>Class:</i> MCAR	Constant Imputation: Treated as uniform dropout, filled with "Unknown".

Variable	Issue, Evidence & Missing Class	Remediation Strategy
gps_lat	Completeness: 36.21% missing, correlates with remote areas. <i>Class:</i> MNAR	Impute & Flag: Created a binary indicator flag before imputing numerical blanks with median.
gps_lon	Completeness: 36.21% missing. <i>Class:</i> MNAR	Impute & Flag: Utilized identical Missing GPS flag, then median-imputed.
reg_id	Consistency: Mixed cases and accidental spaces. <i>Class:</i> N/A	Entity Resolution: Cleaned text to force uppercase formatting for robust group splitting.
manual_score	Timeliness: Target Leakage (correlation 0.981). <i>Class:</i> N/A	Deletion: Dropped completely to prevent artificial offline AUC inflation.
settlement	Timeliness: Target Leakage ("reversed" = 100% fraud). <i>Class:</i> N/A	Deletion: Deleted, as it is strictly unavailable at live prediction.

Section 2. Feature Dictionary

Every engineered feature strictly follows the "as-of" rule: calculations are restricted to historical data occurring strictly before the scoring timestamp t (2026-08-01 00:00:00).

Family 1: Activity and Money Patterns (RFM)

- **recency_days:** Days since the applicant's last transaction. *Window:* History prior to scoring. *Upstream:* ts, customer_id, written at server authorization. *Fairness Note:* None.
- **frequency:** Total historical transactions. *Window:* History prior to scoring. *Upstream:* txn_id, customer_id, written at authorization. *Fairness Note:* Reflects income users who transact less frequently.
- **monetary_out:** Total absolute sum of money spent (debits). *Window:* History prior to scoring. *Upstream:* amount, customer_id, written at authorization. *Fairness Note:* Naturally skews favorably toward wealthier populations.
- **monetary_in:** Total sum of incoming funds (credits). *Window:* History prior to scoring. *Upstream:* amount, customer_id, written at authorization. *Fairness Note:* Reflects alternative income streams.
- **log_amt:** Logarithmic adjustment applied to transaction amounts. *Window:* Per transaction prior to scoring. *Upstream:* amount, written at authorization. *Fairness Note:* Safely compresses heavy-tailed distributions to prevent skew from massive outlier transfers.

Family 2: Ratios and Proportions

- **out_in_ratio:** Ratio of money spent versus received (+1.0 denominator buffer). *Window:* Lifetime history prior to scoring. *Upstream:* amount, customer_id, written at authorization. *Fairness Note:* Gig workers reliant on remittances might exhibit skewed ratios; must not be viewed inherently as high risk.

- **cashout_ratio**: Percentage of transactions explicitly flagged as a cash withdrawal. *Window*: Lifetime history prior to scoring. *Upstream*: `txn_type`, `customer_id`, written at authorization. *Fairness Note*: Unbanked populations rely heavily on physical agents; penalizing this risks structural redlining.

Family 3: Network Stability

- **n_counterparty**: Count of distinct counterparties. *Window*: Lifetime history prior to scoring. *Upstream*: `counterparty`, `customer_id`, written at authorization. *Fairness Note*: Elderly or geographically remote individuals naturally interact with fewer people; weighting heavily risks penalizing smaller digital footprints.

Family 4: Habits, Timing, and Cyclical Flags

- **gps_missing**: Binary flag capturing unrecorded GPS. *Window*: Per transaction. *Upstream*: `gps_lat`, written at mobile authorization. *Fairness Note*: Treating missing GPS solely as a risk flag punishes applicants simply for living up-country.
- **payday_dist**: Days between a transaction and a standard Kenyan payday (1st or 28th). *Window*: Per transaction. *Upstream*: `ts`, written at authorization. *Fairness Note*: Inherently assumes formal employment; unfair to farmers or day-laborers.
- **hour**: Extracted hour of the day (023). *Window*: Per transaction. *Upstream*: `ts`, written at authorization. *Fairness Note*: Strictly penalizing late-night activity could discriminate against night-shift workers.
- **hr_sin** & **hr_cos**: Transformations mapping hours onto a continuous unit circle. *Window*: Per transaction. *Upstream*: `ts`, written at authorization. *Fairness Note*: None.
- **dow**: Raw day of the week. *Window*: Per transaction. *Upstream*: `ts`, written at authorization. *Fairness Note*: None.
- **dow_sin**: Transformation mapping days into a 7-day loop. *Window*: Per transaction. *Upstream*: `ts`, written at authorization. *Fairness Note*: Forcing Monday-to-Friday logic risks cultural bias regarding days of rest.
- **dow_cos**: Transformation mapping days into a 7-day loop cosine coordinate. *Window*: Per transaction. *Upstream*: `ts`, written at authorization. *Fairness Note*: None.

Section 3. Leakage Assessment Memo

During initial pipeline validation, our evaluation produced a false ROC-AUC score of 1.0000. Upon removing leaky fields, the true ROC-AUC dropped to an honest 0.6250, demonstrating a massive +0.3750 inflation caused by future information.

Leak 1: `manual_review_score`

- **Location & Mechanism (Cause 6)**: Found in the raw transaction extract. This falls under *Leakage from the data-collection process* (Cause 6). The score is actively written by the collections and review team *after* a transaction is investigated and flagged. The model read the final answer key during training.

- **Quantified Impact:** Contributed to the +0.3750 offline ROC-AUC inflation, tricking cross-validation.
- **Governance Recommendation:** Implement a mandatory **lineage review** for all new features before entering the data warehouse.

Leak 2: `settlement_status`

- **Location & Mechanism (Cause 6):** Found in the transaction categorical attributes. This represents *Leakage from the data-collection process* (Cause 6). Status updates occur long after initiation (e.g., "reversed" exhibits 100% fraud).
- **Quantified Impact:** Provided near-perfect class separation, driving the false 1.0000 ROC-AUC.
- **Governance Recommendation:** Adopt an enterprise **Feature Store** with strict *point-in-time correctness* guarantees to ensure historical snapshots match live availability.

Section 4. Reproducible Machine Learning Pipeline

Wrapping every preprocessing step inside this pipeline makes data leakage structurally impossible by recalculating imputation medians and vocabulary frequencies strictly on each fold's training slice.

Architectural Design Decisions

1. **Numerical Preprocessing:** Passed to a `SimpleImputer` using the training median with `add_indicator=True` to preserve the MNAR target signal. Transformed via `Yeo-Johnson PowerTransformer` and a `RobustScaler`.
2. **Categorical Preprocessing:** Imputed with an "Unknown" sentinel, then passed to a `OneHotEncoder` (`handle_unknown="infrequent_if_exist"`, `min_frequency=50`).
3. **Group-Aware Splitting:** Evaluated using `GroupKFold` (5 splits) grouped by resolved `customer_id` to prevent multi-SIM card leakage across folds.
4. **Estimator & Serialization:** The `LogisticRegression` (`max_iter=2000`) pipeline achieves a cross-validated ROC-AUC of **0.6250 (+/- 0.0026)** and is serialized into a `.joblib` artifact for seamless deployment.

```
import joblib
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, RobustScaler, PowerTransformer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score, GroupKFold

NUMERICAL_COLS = ["amount_abs", "log_amt", "hr_sin", "hr_cos", "dow_sin", "dow_cos",
                  "payday_dist", "gps_missing", "recency_days", "frequency",
                  "monetary_out", "monetary_in", "n_counterparty", "cashout_ratio",
                  "out_in_ratio"]
CATEGORY_COLS = ["txn_type", "region", "segment"]

def build_model(num_cols, cat_cols):
    num = Pipeline([
```

```

        ("impute", SimpleImputer(strategy="median", add_indicator=True)),
        ("power", PowerTransformer(method="yeo-johnson")),
        ("scale", RobustScaler()),
    ])
    cat = Pipeline([
        ("impute", SimpleImputer(strategy="constant", fill_value="Unknown")),
        ("onehot", OneHotEncoder(handle_unknown="infrequent_if_exist",
                                  min_frequency=50)),
    ])
    prep = ColumnTransformer([("num", num, num_cols), ("cat", cat, cat_cols)])
    return Pipeline([("prep", prep), ("clf", LogisticRegression(max_iter=2000))])

clean_model = build_model(NUMERICAL_COLS, CATEGORY_COLS)

y = df["is_fraud"].values
groups = df["customer_id"].values
cv = GroupKFold(n_splits=5)

scores = cross_val_score(clean_model, df, y, cv=cv, groups=groups, scoring="
    roc_auc")
print(f"Honest ROC-AUC: {scores.mean():.4f} (+/- {scores.std():.4f})")

clean_model.fit(df, y)
joblib.dump(clean_model, "credit_scoring_pipeline_v1.joblib")

```